

Todos los algoritmos de la materia

Inteligencia Computacional · FICH-UNL · cada ecuación en orden, y debajo qué dice en palabras

Los resúmenes de cada unidad explican **por qué** funciona cada pieza, con las derivaciones y los ejemplos numéricos. Este archivo es sólo el **qué**: los algoritmos, uno atrás del otro, en el orden en que se ejecutan. Cada ecuación con un párrafo debajo que dice qué significa. Sirve para repasar de corrido y para desarrollar en la pizarra.

#	Algoritmo	Supervisado	Entrenamiento
1	Perceptrón simple — corrección de error	sí	iterativo
2	Perceptrón simple — gradiente (LMS)	sí	iterativo
3	Perceptrón multicapa — back-propagation	sí	iterativo
4	Base radial	mixto	dos fases
5	Hopfield	no	directo
6	BPTT	sí	iterativo
7	SOM	no	iterativo
8	LVQ1	sí	iterativo

1. Perceptrón simple — corrección de error

IDEA DE FONDO — en qué consiste

Una sola neurona que traza una **recta** (un hiperplano) y dice de qué lado cayó cada punto. Tiene dos partes que se repiten patrón por patrón: **calcular la salida** y, **sólo si se equivocó, mover la recta**. La imagen para recordarlo: la recta está mal puesta, entra un punto que quedó del lado equivocado, y la recta **gira un poco hacia ese punto**. Si el punto ya estaba bien, no pasa nada. Repetido muchas veces, la recta se acomoda hasta no equivocarse con ninguno — y eso está garantizado **sólo si los datos se pueden separar con una recta**.

Datos de partida

$$\{(\mathbf{x}(n), d(n))\}, \quad x_0 = -1$$

Pares entrada–salida deseada. La componente $x_0 = -1$ es fija y existe para que el umbral entre como un peso más, $w_0 = u$, y no haya que tratarlo aparte.

Salida

$$y = \varphi(\langle \mathbf{w}, \mathbf{x} \rangle)$$

Se hace el producto interno entre los pesos y la entrada, y se le aplica la función de activación. Todo el perceptrón es eso.

$$\langle \mathbf{w}, \mathbf{x} \rangle = 0$$

Los puntos donde el producto interno da cero son la **frontera de decisión**: un hiperplano que parte el espacio de entrada en dos semiplanos.

Adaptación

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\eta}{2} [d(n) - y(n)] \mathbf{x}(n)$$

Si la salida coincide con la deseada, el corchete es cero y los pesos no se tocan. Si no coincide, los pesos se mueven en la dirección de la entrada, con el signo del error: la frontera gira hacia el lado que corrige ese patrón.

Parada

$$d(n) = y(n) \quad \forall n$$

Se termina cuando una pasada completa por el conjunto no produce ninguna corrección. Está garantizado que eso pasa **sólo si el problema es linealmente separable**.

2. Perceptrón simple — regla del gradiente (LMS)

IDEA DE FONDO — en qué consiste

La misma neurona, pero con otra filosofía: en vez de reaccionar sólo cuando se equivoca, **define un número que mide lo mal que está** y se mueve siempre en la dirección que más lo baja. Tres partes: **definir el criterio de error**, **derivarlo** respecto de los pesos, y **moverse en contra de esa derivada**. La imagen: una pelota bajando por una ladera, donde la altura es el error y la posición son los pesos. Es el molde de todos los algoritmos supervisados que vienen después — multicapa, radial y BPTT son este mismo esquema con un gradiente más difícil de calcular.

Mismo modelo de neurona; cambia de dónde sale el ajuste. Aquí no se corrige el error, se minimiza una función.

Criterio

$$\xi(n) = \frac{1}{2} e^2(n), \quad e(n) = d(n) - y(n)$$

Se define una medida de lo mal que está la red para el patrón actual: el error al cuadrado, por comodidad analítica. El $\frac{1}{2}$ está para que se cancele con el 2 que baja al derivar.

Regla del gradiente

$$\Delta \mathbf{w}(n) = -\mu \nabla_{\mathbf{w}} \xi(n)$$

Los pesos se mueven en la dirección **opuesta** al gradiente, que es la de máximo crecimiento del error: se baja por la superficie de error. μ dice qué tan grande es el paso.

LMS (Widrow-Hoff)

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n) \mathbf{x}(n)$$

Resolviendo el gradiente para este criterio queda esto: paso, por error, por entrada. Es la misma forma que la corrección de error, y de hecho coinciden con $\eta = 4\mu$.

Acá aparece por primera vez la estructura que se repite en todo lo que sigue: **velocidad de aprendizaje, por error, por entrada**.

3. Perceptrón multicapa — back-propagation

IDEA DE FONDO — en qué consiste

Varias capas de neuronas, cada una alimentando a la siguiente. Por cada patrón se hacen **tres cosas en este orden**: **ir hacia adelante** calculando las salidas de capa en capa, **medir el error** en la última, y **volver hacia atrás repartiendo la culpa**. El problema que resuelve, y es todo el tema: una neurona oculta **no tiene salida deseada**, así que no se sabe cuánto se equivocó. La solución: su error se arma **juntando los errores de todas las neuronas que ella alimenta**, cada uno pesado por la conexión que las une. Eso es lo que significa «retropropagar»: el error viaja hacia atrás por los mismos cables que la señal viajó hacia adelante.

Arquitectura

$$\mathbf{v}^{(p)} = \mathbf{W}^{(p)} \mathbf{y}^{(p-1)} \quad y_j^{(p)} = \varphi(v_j^{(p)})$$

Cada capa toma la salida de la anterior, la multiplica por su matriz de pesos y le aplica la activación. $\mathbf{W}^{(p)}$ es de $M_p \times (M_{p-1} + 1)$: la columna extra es la del sesgo.

$$\varphi(v) = \frac{2}{1 + e^{-bv}} - 1$$

La sigmoide **simétrica**, que va de -1 a $+1$. Se usa ésta y no la logística porque las salidas centradas en cero hacen que el aprendizaje sea más parejo.

Error

$$e_j(n) = d_j(n) - y_j(n) \quad \xi(n) = \frac{1}{2} \sum_{j=1}^M e_j^2(n)$$

El error de cada neurona de salida, y el error instantáneo de la red como la suma de todos al cuadrado.

Regla del gradiente

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

Lo mismo que en el perceptrón, pero ahora el peso puede estar en cualquier capa y no hay salida deseada para las ocultas. Eso es lo que hay que resolver.

$$\frac{\partial \xi}{\partial w_{ji}} = \frac{\partial \xi}{\partial e_j} \frac{\partial e_j}{\partial y_j} \frac{\partial y_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}}$$

La cadena a derivar. Los dos primeros factores dan el error, el tercero es la derivada de la activación y el cuarto es la entrada que llega por esa conexión.

La derivada de la activación

$$\varphi'(v_j) = \frac{1}{2}(1 + y_j)(1 - y_j)$$

Se escribe en función de la **salida**, no del campo local: como y_j ya está calculado del paso hacia adelante, la derivada sale gratis. El $\frac{1}{2}$ va (con b general sería $b/2$).

Los deltas

$$\delta_j(n) = -\frac{\partial \xi}{\partial y_j} \varphi'(v_j)$$

Se le pone nombre a todo lo que pasa **dentro** de la neurona, para separarlo de lo que pasa en la conexión.

$$\delta_j^{III} = \frac{1}{2} e_j (1 + y_j^{III})(1 - y_j^{III})$$

En la capa de salida hay salida deseada, así que el delta es directo: error por derivada de la activación.

$$\delta_j^{II} = \left[\sum_k \delta_k^{III} w_{kj}^{III} \right] \frac{1}{2} (1 + y_j^{II})(1 - y_j^{II})$$

En una capa oculta no hay salida deseada, así que el error se trae de atrás: se juntan los deltas de la capa siguiente, cada uno pesado por la conexión que lo une con esta neurona. **El corchete reemplaza al error**, y el resto de la fórmula es idéntico.

Actualización

$$\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n) \quad w_{ji}(n+1) = w_{ji}(n) + \Delta w_{ji}(n)$$

La regla vale para cualquier capa: paso, por delta de la neurona a la que llega la conexión, por la señal que entra por ella. Lo único que cambia entre capas es cómo se calculó el delta.

$$\Delta w_{j0}^{(p)} = \mu \delta_j^{(p)} (-1)$$

El peso de sesgo se ajusta igual, con la entrada fija -1 .

4. Redes de base radial

IDEA DE FONDO — en qué consiste

Cambiar de representación para que el problema se vuelve fácil. Tiene **dos fases con criterios distintos**: primero, **sin mirar las etiquetas**, se ubican unas cuantas «burbujas» gaussianas donde hay datos; después, **mirando las etiquetas**, se decide cuánto pesa cada burbuja en la salida. La imagen: cada neurona oculta es un detector local que se enciende cuando la entrada cae cerca de su centro y se apaga si está lejos — al revés de la sigmoide, que responde a todo un semiespacio. Y la clave del método: una vez fijadas las burbujas, **la salida es una combinación lineal**, así que la segunda mitad es un problema lineal, sin capas ocultas que derivar.

Dos fases con criterios distintos: la primera no supervisada ubica las funciones, la segunda supervisada las combina.

Arquitectura

$$\varphi_j(\mathbf{x}_\ell) = e^{-\|\mathbf{x}_\ell - \mu_j\|^2 / 2\sigma_j^2}$$

Cada neurona oculta responde según **qué tan cerca** está la entrada de su centro μ_j : máximo si coincide, y cae con la distancia a una velocidad que fija σ_j . Es una activación local, al revés que la sigmoide, que responde a todo un semiespacio.

$$y_k(\mathbf{x}_\ell) = \sum_{j=1}^M w_{kj} \varphi_j(\mathbf{x}_\ell)$$

La salida es una **combinación lineal** de esas respuestas. Y ahí está la clave del método: fijadas las funciones radiales, la salida es lineal en los pesos, así que la segunda fase es un problema fácil.

Fase 1 — centros, no supervisada (*k*-medias)

$$J = \sum_j \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \mu_j\|^2$$

El criterio: que cada dato esté lo más cerca posible del centro de su grupo. Se minimiza alternando dos pasos.

$$\ell \in C_j \iff \|\mathbf{x}_\ell - \mu_j\|^2 < \|\mathbf{x}_\ell - \mu_i\|^2 \quad \forall i \neq j$$

Reasignación: cada dato se asigna al centro más cercano.

$$\mu_j = \frac{1}{|C_j|} \sum_{\ell \in C_j} \mathbf{x}_\ell$$

Recálculo: cada centro se muda al promedio de los datos que le tocaron. Se repite hasta que nadie cambia de grupo.

$$j^* = \arg \min_j \|\mathbf{x}_\ell - \mu_j(n)\| \quad \mu_{j^*}(n+1) = \mu_{j^*}(n) + \eta(\mathbf{x}_\ell - \mu_{j^*}(n))$$

La versión **en línea**, patrón por patrón: gana el centro más cercano y se corre un poco hacia el dato. Es la misma idea que el SOM pero sin vecindad.

Fase 2 — pesos, supervisada

$$e_k(n) = y_k(n) - d_k(n) \quad \xi(n) = \frac{1}{2} \sum_k e_k^2(n)$$

Recién acá aparecen las salidas deseadas.

$$\frac{\partial \xi}{\partial w_{kj}} = e_k(n) \varphi_j(n)$$

El gradiente es sencillo porque la salida es lineal en los pesos: no hay ninguna derivada de activación que arrastrar.

$$w_{kj}(n+1) = w_{kj}(n) - \eta e_k(n) \varphi_j(n)$$

Error por activación de la neurona oculta. Es LMS otra vez, con las φ_j haciendo de entradas.

5. Hopfield

IDEA DE FONDO — en qué consiste

Una memoria que se consulta **por contenido**: le das un dato roto y te devuelve el dato entero. Tiene **dos fases**: **almacenar**, que es una sola cuenta y no itera, y **recuperar**, que itera hasta quedarse quieta. La idea de fondo: el recuerdo **no se guarda en ningún lugar**, se guarda como **acuerdos entre las partes del recuerdo** — para cada par de neuronas, si suelen coincidir o suelen oponerse. Por eso están **todas conectadas con todas**, en una sola capa. Y recuperar es **una votación ponderada que se repite**: se sortea una neurona, las demás le dicen qué valor debería tener, **el peso del cable es cuánto vale cada voto**, gana la mayoría, y se repite hasta que nadie cambia.

Datos de partida

$$\mathbf{x}_k^* \in \{-1, +1\}^N, \quad k = 1, \dots, P$$

P patrones para guardar, cada uno un vector de N componentes que valen -1 o $+1$. Esos N lugares son las N neuronas.

Fase 1 — Almacenamiento

$$w_{ji} = \frac{1}{N} \sum_{k=1}^P x_{kj}^* x_{ki}^* \quad \forall i \neq j$$

Para cada par de neuronas se recorren los patrones multiplicando el valor que tiene una por el valor que tiene la otra. Como son ± 1 , cada producto da $+1$ si coinciden y -1 si se oponen: el peso mide **en cuántos patrones coincidieron menos en cuántos difirieron**.

$$w_{jj} = 0 \quad \forall j$$

Ninguna neurona se conecta consigo misma. Se impone: la fórmula daría P/N y esa neurona se haría caso a sí misma por encima de todas las demás.

$$w_{ji} = w_{ij}$$

La conexión entre dos neuronas es una sola. Con la regla de arriba se cumple solo, porque el producto no distingue el orden.

Fase 2 — Recuperación

$$\mathbf{y}(0) = \mathbf{x}$$

El patrón incompleto o con ruido se escribe directamente como estado inicial de las neuronas.

$$j^* = \text{rnd}(N)$$

Se sortea cuál neurona se actualiza en este paso.

$$v_{j^*}(n) = \sum_{i=1}^N w_{j^*i} y_i(n-1)$$

El campo local: el valor actual de cada una de las demás, multiplicado por el peso que la une con la elegida, todo sumado. Es una votación ponderada, donde el peso es cuánto vale cada voto.

$$y_{j^*}(n) = \text{sgn}(v_{j^*}(n)), \quad \text{sgn}(x) = \begin{cases} +1 & x > 0 \\ y_{j^*}(n-1) & x = 0 \\ -1 & x < 0 \end{cases}$$

El nuevo valor es el signo de ese total: gana la mayoría. En el empate la neurona conserva lo que tenía.

$$y_j(n) = y_j(n-1) \quad \forall j \neq j^*$$

Las demás quedan como estaban: se toca **una sola** por paso.

$$\mathbf{y}(n) = \dots = \mathbf{y}(n-N) \implies \text{fin}$$

Se corta tras N actualizaciones seguidas sin cambios. Recién ahí nada puede moverse después, porque a cada neurona le entraría lo mismo que ya le entró.

Convergencia

$$E(\mathbf{y}) = -\frac{1}{2} \sum_j \sum_i w_{ji} y_i y_j$$

Cada par de neuronas aporta un término que baja la energía cuando el par está como su peso “quiere”. El $\frac{1}{2}$ compensa que la doble suma cuenta cada par dos veces.

$$\Delta E = -(y_{j^*}(n) - y_{j^*}(n-1)) v_{j^*}(n)$$

Al actualizar una sola neurona, los únicos términos que se mueven son los que la contienen. **Esto vale porque los pesos son simétricos**: si no, el par (i, j) aportaría dos términos distintos.

$$\Delta E = 2 y_{j^*}(n-1) v_{j^*}(n) < 0 \quad \text{si cambió} \quad \Delta E = 0 \quad \text{si no cambió}$$

Si la neurona se dio vuelta fue porque su valor viejo no coincidía con el signo del campo: tienen signos distintos, el producto es negativo y la energía bajó.

$$\boxed{\Delta E \leq 0} \wedge 2^N < \infty \implies \text{converge en pasos finitos}$$

La energía nunca sube, está acotada por abajo y hay finitos estados: no puede bajar para siempre.

$$E(-\mathbf{y}) = E(\mathbf{y}) \quad P_{\max} = \frac{N}{2 \ln N}$$

Dar vuelta todas las neuronas no cambia la energía: por cada memoria guardada queda su negativo como estado espúreo. Y la capacidad crece más lento que N , mientras los pesos crecen como N^2 .

6. BPTT

IDEA DE FONDO — en qué consiste

Entrenar una red recurrente con back-propagation, **desenrollando el tiempo**: si la red se realimenta durante T instantes, se la dibuja como T copias en fila y se la trata como una red profunda. Tres partes: **ida** por toda la secuencia guardando todo, **medir los errores**, y **vuelta en el tiempo** acumulando gradientes. Las dos diferencias con el multica, que son de donde salen todas las preguntas: **el tiempo hace de profundidad**, y **las copias comparten los mismos pesos** — por eso los aportes de todos los instantes **se suman**.

sobre el mismo peso en vez de ajustar cada capa por separado.

Datos de partida

$$\{(\mathbf{x}_t, \mathbf{d}_t)\}_{t=0}^{T-1}$$

Una secuencia, con salida deseada en los instantes que la tengan. No todos necesitan tenerla.

Inicialización

$$\mathbf{W}^I, \mathbf{W} \sim \text{al azar, chicos} \quad \mathbf{y}_{-1} = \mathbf{0} \quad \Delta \mathbf{W} = \Delta \mathbf{W}^I = \mathbf{0}$$

Pesos chicos para que las neuronas no arranquen saturadas con $\varphi' \approx 0$. Estado previo nulo y acumuladores en cero.

Paso hacia adelante

$$\mathbf{v}_t = \mathbf{W}^I \mathbf{x}_t + \mathbf{W} \mathbf{y}_{t-1} \quad \mathbf{y}_t = \varphi(\mathbf{v}_t)$$

Lo que entra tiene dos partes: la entrada de ese instante y la salida de la propia red en el anterior. Ésa es toda la recurrencia, y la **misma** $\mathbf{W}$ se usa en todos los instantes.

$$\text{guardar } \mathbf{v}_t, \mathbf{y}_t \quad \forall t$$

La vuelta va a necesitar $\varphi'(\mathbf{v}_t)$ de cada instante. Es la diferencia práctica más grande con el multicapa, y la razón de que la memoria crezca con T .

Error

$$\mathbf{e}_t = \mathbf{y}_t - \mathbf{d}_t \quad E = \frac{1}{2} \sum_t \sum_k e_{t,k}^2$$

El error de la secuencia es la suma de los errores de todos los instantes. Los que no tienen salida deseada aportan cero.

Gradiente — forma derivada

$$\delta_{t,j} = e_{t,j} \varphi'(v_{t,j})$$

En el instante donde se midió el error, el delta es directo: igual que la capa de salida del multicapa.

$$\delta_{\tau,j} = \left(\sum_k w_{jk} \delta_{\tau+1,k} \right) \varphi'(v_{\tau,j}) \quad \tau < t$$

Hacia atrás en el tiempo, el delta se arrastra: los deltas del instante siguiente, pesados por las conexiones, por la derivada de la activación de esta neurona en este instante. **El tiempo hace de profundidad.**

$$\frac{\partial E_t}{\partial w_{ji}} = \sum_{\tau=t}^0 \delta_{\tau,j} y_{\tau-1,i} \quad \frac{\partial E_t}{\partial w_{ji}^I} = \sum_{\tau=t}^0 \delta_{\tau,j} x_{\tau,i}$$

El error de un instante se reparte sobre **todos** los anteriores, porque el mismo peso participó en todos, y los aportes se **suman**. Para los pesos de entrada los deltas son los mismos: cambia sólo el último factor.

Gradiente — forma optimizada

$$\delta_T^* = \mathbf{0} \quad \delta_t^* = [\mathbf{e}_t + \mathbf{W}^T \delta_{t+1}^*] \odot \varphi'(\mathbf{v}_t)$$

El delta acumulado junta el error medido en ese instante con todo lo arrastrado de los posteriores, ya sumado. Se recorre de $T - 1$ a 0, al revés que la propagación. La **transpuesta** es lo que distingue la ida de la vuelta.

$$\Delta \mathbf{W} += \delta_t^* \mathbf{y}_{t-1}^T \quad \Delta \mathbf{W}^I += \delta_t^* \mathbf{x}_t^T$$

En el mismo barrido se acumulan los dos gradientes. Da **exactamente el mismo** gradiente que la forma derivada, reorganizado.

Actualización

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \Delta \mathbf{W} \quad \mathbf{W}^I \leftarrow \mathbf{W}^I - \eta \Delta \mathbf{W}^I$$

Los pesos se tocan **al final de la secuencia**. Si se actualizaran en el medio del barrido, los deltas que faltan usarían una $\mathbf{W}$ distinta de la que se usó hacia adelante.

Truncado

$$\tau \geq \max(0, t - P) \quad O(T^2) \rightarrow O(T)$$

Truncar limita la vuelta a P instantes: se pierden las dependencias largas. **Truncar cambia el gradiente; optimizar con δ^* no.**

Arquitecturas con retardo

$$\tilde{\mathbf{x}}_t = (\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-D})$$

TDNN: la entrada se amplía con sus retardos y la red vuelve a ser hacia adelante. Toda la memoria está en la entrada, no en la red.

$$\mathbf{y}_t = \varphi(\mathbf{W}^I \mathbf{x}_t + \mathbf{W}^C \mathbf{c}_t), \quad \mathbf{c}_t = \mathbf{y}_{t-1}^{\text{oculta}} \text{ (Elman)} \quad \mathbf{c}_t = \mathbf{y}_{t-1}^{\text{salida}} \text{ (Jordan)}$$

La capa de contexto guarda la salida anterior —de la oculta en Elman, de la salida en Jordan— y se trata como una entrada más con valores congelados. Con eso se entrena con back-propagation común: es BPTT truncada a un solo paso.

7. SOM

IDEA DE FONDO — en qué consiste

Acomodar unos pocos prototipos para que representen a muchos datos, y **además queden ordenados entre sí**. Dos partes por cada patrón: **competir** (gana el prototipo más cercano) y **adaptar** (el ganador se corre un poco hacia el dato). La imagen clave: los pesos de cada neurona **viven en el mismo espacio que los datos**, así que son un punto que se va mudando hacia donde hay datos. Y lo que lo hace un *mapa* y no una simple cuantización: **cuando el ganador se mueve, arrastra a sus vecinos de la grilla**. Por eso neuronas contiguas terminan representando zonas contiguas del espacio de entrada.

Datos de partida

$$\{\mathbf{x}(n)\} \subset \mathbb{R}^M, \quad \text{sin etiquetas}$$

Sólo vectores de entrada. El entrenamiento es **no supervisado**: la red se organiza a partir de cómo están distribuidos los datos.

Arquitectura

$$\mathbf{w}_j \in \mathbb{R}^M, \quad j = 1, \dots, N$$

Los pesos de cada neurona **viven en el mismo espacio que las entradas**: son un punto del espacio de datos, no coeficientes. Por eso tiene sentido la distancia entre una entrada y una neurona.

Las neuronas están además en una grilla, que no tiene nada que ver con $\mathbb{R}^M$: define **quién es vecino de quién**, y es lo único que la topología aporta.

Inicialización

$$\mathbf{w}_j(0) \sim \text{al azar, chicos y distintos entre sí}$$

Distintos obligatoriamente: dos neuronas que arrancan en el mismo punto ganan y se mueven siempre juntas, y nunca se separan.

Competencia

$$j^*(n) = G(\mathbf{x}(n)) = \arg \min_{\forall j} \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

Se presenta una entrada y gana la neurona más cercana en el espacio de datos, midiendo con distancia euclídea.

Vecindad

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

La influencia sobre cada neurona decae con su distancia **en la grilla** a la ganadora: la de al lado se mueve casi tanto como ella, una lejana casi nada. Con entorno uniforme, en cambio, es $\beta(n)$ adentro del radio y cero afuera.

Adaptación

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n)(\mathbf{x}(n) - \mathbf{w}_j(n)) & y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & y_j \notin \Lambda_G(n) \end{cases}$$

La ganadora y sus vecinas se mueven hacia la entrada: el paréntesis es el vector que va de la neurona al dato, y η dice qué fracción de ese camino se recorre. Las de afuera del entorno no se tocan.

Que se muevan **las vecinas de la grilla** es lo único que separa un SOM de una cuantización vectorial cualquiera: es lo que hace que neuronas contiguas terminen representando zonas contiguas del espacio de entrada.

Decaimiento — las tres etapas

$$\eta(n) \downarrow, \quad \Lambda_G(n) \downarrow$$

Etapas	Λ_G	η	Épocas
Ordenamiento	$\approx$ medio mapa	0,9 – 0,7	500 – 1000
Transición	$\rightarrow 1$, lineal	$\rightarrow 0,1$	≈ 1000
Ajuste fino	0	0,1 – 0,01, cte.	≈ 3000

Primero se ordena el mapa a lo grande con entorno amplio y pasos largos; después se afina con entorno nulo y pasos cortos. No hay error que mirar: se corta por cantidad de épocas.

8. LVQ1

IDEA DE FONDO — en qué consiste

Lo mismo que el SOM pero **con etiquetas y con castigo**. Tres partes: **competir**, **ver si el ganador era de la clase correcta**, y **premiar o castigar**. Si acertó, el prototipo se acerca al dato; si se equivocó, **se aleja**. Eso es todo lo que cambia, y cambia el objetivo: al SOM le importa cubrir bien el espacio, a LVQ le importa **dónde queda la frontera entre clases**. Por eso acá no hay vecindad: se mueve sólo el ganador.

Datos de partida

$$\{(\mathbf{x}(n), d(n))\}, \quad \mathcal{C}(i) \text{ asignada a cada prototipo}$$

Acá **sí** hay etiquetas: LVQ es supervisado. Y cada prototipo tiene su clase asignada de antemano.

Ganador

$$c(n) = \arg \min_i \{\|\mathbf{x}(n) - \mathbf{m}_i(n)\|\}$$

Igual que en el SOM: gana el prototipo más cercano.

Signo

$$s(c, d, n) = \begin{cases} +1 & \mathcal{C}(c(n)) = d(n) \\ -1 & \mathcal{C}(c(n)) \neq d(n) \end{cases}$$

Acá está toda la diferencia con el SOM: se compara la clase del prototipo que ganó con la clase verdadera del dato.

Adaptación

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(c, d, n) \alpha [\mathbf{x}(n) - \mathbf{m}_c(n)]$$

El ganador se mueve **hacia** el dato si acertó la clase, y se **aleja** si se equivocó. El castigo es lo que no existe en el SOM, y es lo que empuja la frontera entre clases a su lugar.

$$\mathbf{m}_i(n+1) = \mathbf{m}_i(n) \quad \forall i \neq c$$

Se mueve **sólo el ganador**: no hay vecindad. A LVQ no le interesa la topología, le interesa la frontera de decisión.

Forma combinación

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)] \mathbf{m}_c(n) + s(n)\alpha(n) \mathbf{x}(n)$$

La misma regla reescrita: el prototipo nuevo es una combinación del viejo y del patrón que acaba de entrar. Escrita así se ve el problema del α constante.

LVQ1-O — el α óptimo

$$\alpha_c(n) = [1 - s(n)\alpha_c(n)] \alpha_c(n-1)$$

La condición que se impone: que el patrón anterior siga pesando lo mismo que el actual después de este paso. Con α constante no pasa — cada patrón viejo arrastra un factor más, y el prototipo termina dependiendo casi sólo del final del archivo, que es un orden arbitrario.

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)}$$

Despejando queda esto: un α propio de cada prototipo, que decrece solo cuando acierta y crece cuando se equivoca. **No debe superar** $\alpha > 1$.

9. Protocolo de evaluación

IDEA DE FONDO — en qué consiste

Contestar «¿qué tan bien anda esto?» sin engañarse. Dos partes: **cómo partir los datos** (entrenamiento para ajustar los pesos, monitoreo para decidir cuándo parar y qué arquitectura usar, prueba para medir **una sola vez** al final) y **con qué número medir**. La trampa que hay que saber nombrar: si elegís la cantidad de épocas o la arquitectura mirando el conjunto de prueba, ese número **ya no mide generalización**, porque lo usaste para decidir.

No es un algoritmo de aprendizaje, pero es procedimiento y se pregunta igual.

Validación cruzada

$$\bar{\varepsilon} = \frac{1}{n} \sum_{j=1}^n \varepsilon_j \quad \sigma_{\varepsilon}^2 = \frac{1}{n} \sum_{j=1}^n (\varepsilon_j - \bar{\varepsilon})^2$$

Se entrena n veces, cada una dejando afuera una partición distinta, y se promedian los errores. **La varianza se**

reporta siempre junto con la media: dice si el resultado es estable o si dependió de qué patrones tocaron. Cada partición reinicializa los pesos de cero.

Matriz de confusión

$$N_{\oplus} = t_{\oplus} + f_{\ominus} \quad N_{\ominus} = f_{\oplus} + t_{\ominus} \quad N = N_{\oplus} + N_{\ominus}$$

Los totales por fila son los que **realmente** pertenecen a cada clase. Ojo con la lectura: $f_{\ominus}$ es “dijo negativo y erró”, o sea que era positivo.

$$s^+ = \frac{t_{\oplus}}{N_{\oplus}} \quad s^- = \frac{t_{\ominus}}{N_{\ominus}} \quad p = \frac{t_{\oplus}}{t_{\oplus} + f_{\oplus}} \quad a = \frac{t_{\oplus} + t_{\ominus}}{N}$$

Sensibilidad y especificidad miran una fila cada una; la precisión mira una columna; la exactitud, todo. Un clasificador que dice siempre lo mismo engaña a varias de ellas: por eso nunca se reporta una sola.

$$F_1 = \frac{2t_{\oplus}}{2t_{\oplus} + f_{\oplus} + f_{\ominus}} = 2 \frac{s^+ p}{s^+ + p} \quad G = \sqrt{s^+ p}$$

Combinan sensibilidad y precisión en un número: F_1 es su media armónica, G la geométrica.

$$e = 1 - a \quad \delta_e = \frac{e_r - e}{e_r}$$

La mejora relativa respecto de un sistema de referencia: pasar de 98 % a 99 % de exactitud es reducir el error a la mitad, y pasar de 50 % a 51 % no es lo mismo aunque el salto de exactitud sea igual.

Medidas para predicción

$$e_i = y_i - \hat{y}_i \quad \tilde{\epsilon}_A = \frac{1}{N} \sum_i |e_i| \quad \tilde{\epsilon}_S = \frac{1}{N} \sum_i e_i^2 \quad \tilde{\epsilon}_R = \sqrt{\tilde{\epsilon}_S}$$

No se puede usar la suma de los errores: los signos se compensan. Por eso valor absoluto o cuadrado, y la raíz del cuadrático para volver a las unidades del dato.

Los algoritmos, lado a lado

Algoritmo	Qué ajusta	Criterio	Competitivo	Parada
Perceptrón (corrección)	$\mathbf{w}$	$d - y$	no	sin errores en una pasada
Perceptrón (LMS)	$\mathbf{w}$	$\frac{1}{2} e^2$	no	épocas / error
Multicapa	todas las $\mathbf{W}^{(p)}$	$\frac{1}{2} \sum_j e_j^2$	no	épocas / monitoreo
Base radial fase 1	μ_j	J de k -medias	sí	sin reasignaciones
Base radial fase 2	w_{kj}	$\frac{1}{2} \sum_k e_k^2$	no	épocas / error
Hopfield	w_{ji} (una vez)	— (Hebb)	no	N pasos sin cambios
BPTT	$\mathbf{W}, \mathbf{W}^I$	$\frac{1}{2} \sum_t \sum_k e^2$	no	épocas
SOM	$\mathbf{w}_j$ y vecinas	—	sí	épocas
LVQ1	$\mathbf{m}_c$	acierto de clase	sí	épocas

Las cuatro frases que ordenan todo

1. **Perceptrón, multicapa, RBF fase 2 y BPTT son el mismo esquema:** bajar por el gradiente de un error cuadrático. Lo que cambia es qué tan difícil es calcular ese gradiente.
2. **k -medias, SOM y LVQ son el mismo esquema:** gana el más cercano y se mueve hacia el dato. Lo que cambia es quién más se mueve — nadie, los vecinos de la grilla, o el ganador con signo.
3. **Hopfield es el único que no itera para entrenar:** una cuenta y listo. Y es el único que itera para usarse.
4. **La activación decide la forma de la frontera:** sigmoide, semiespacios; radial, burbujas locales.